Iterative Socket Server

Author: Marcus Nix

Class: CNT4504 Computer Networks & Distributed Processing

Professor: Scott Kelly

Introduction

The purpose of this project was to explore the creation of an Iterative Socket Server and observe the dynamics of client-server interactions. Additionally, this project served as meaningful practice in implementing multithreaded programming techniques. The goal of this project was to successfully establish a client-server connection via two respective Java Classes. The client class was to provide multithreaded capability for 6 unique queries selectable by the user. Additionally, this program provides the capability for the user to specify the socket connection on their device.

In the following section (Client-Server Setup and Configuration), you will find a brief explanation of the APIs used to establish the socket connection, as well as a detailed account on the techniques used to setup, maintain, and end the connection. In the testing section (Testing and Data Collection), a walk-through on how the program was tested during the coding process will be provided, as well as the methods used to obtain and manage the data passed between the client and server. In the final two sections (Data Analysis, and Conclusion), I will summarize my thoughts on the experience gained, and give anecdotes on the data I was able to collect within the scope of the project.

Client-Server Setup and Configuration

The client and server programs contain the same basic structure: an initialization phase at the beginning of the main method, a recursive menu that allows the user to specify which data to collect, and various helper methods that provide transitional functionality between the client and the server. This design structure was chosen via client specifications. The presentation and navigation of the client program specifically was a key deliverable for this service. Both client, and server begin by prompting the user for socket information. The server requests a port number via the java.net.Server, and .ServerSocket APIs. The client requests both the network address, and port number utilizing java.net.Client. Once the socket connection has been established, the client continuously prompts the user for the command they'd like to execute. These commands range from getting the current date and time, network statistics via netstat, and viewing the current processes currently running on the server (among others).

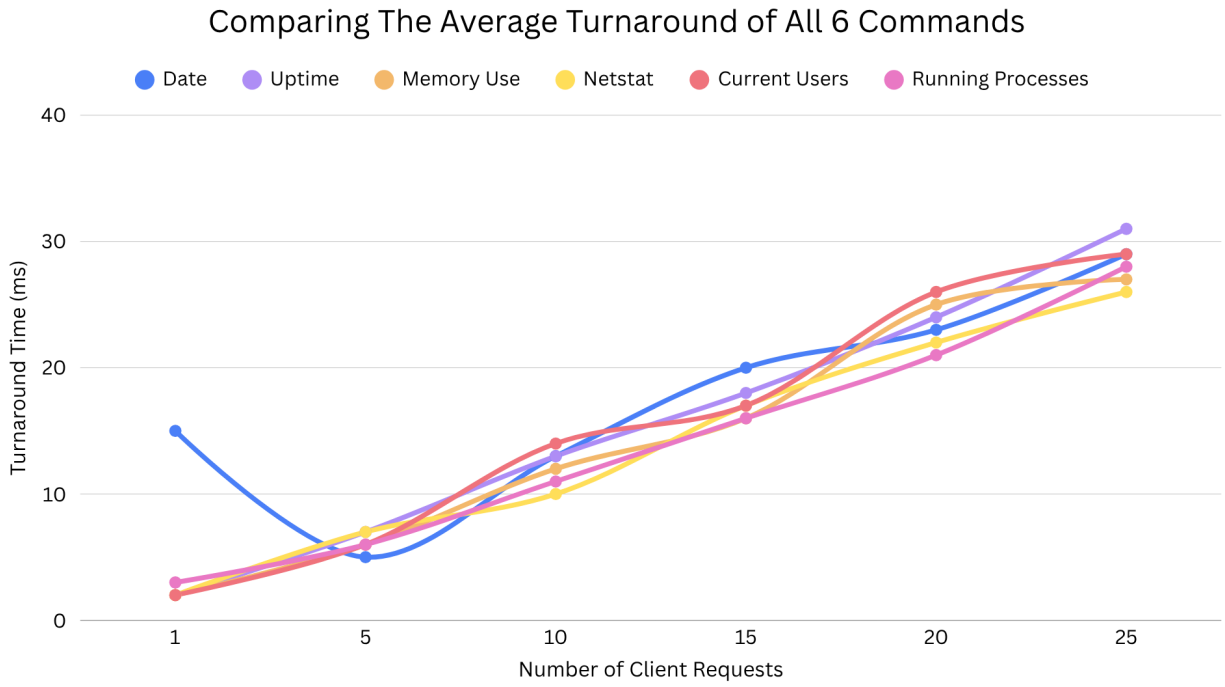
Once the user selects their query, the client then prompts the user for the number of client sessions they would like to create: 1, 5, 10, 15, 20, 25. The chosen value corresponds to the number of open threads that will send that query to the server. Regardless of the option the user chooses, I have developed a single data handler method called “getData” that takes in the user selection information, and transmits that data to the server via data input and output streams. The system and server both have an input stream, and the client and server programs share an output stream between themselves. The data is passed between programs via UTF encoding which provides the data as a string token. Each token prompts the server’s “sendData” method which executes the command specified in the client menu, parses the data into string variables, and returns to each thread. The threads then prompt the client output stream to display the desired information to the user. This information includes the output of the chosen command, the turnaround time for the individual threads, and the average/total turnaround times for the query overall.

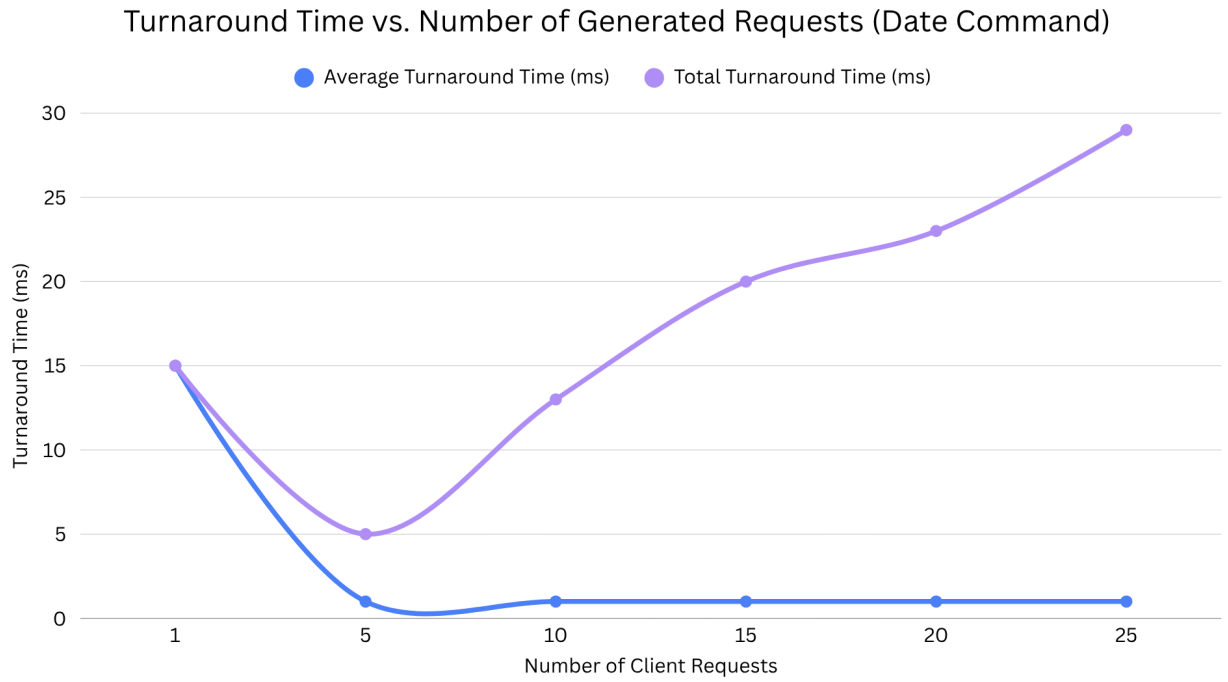
Testing and Data Collection

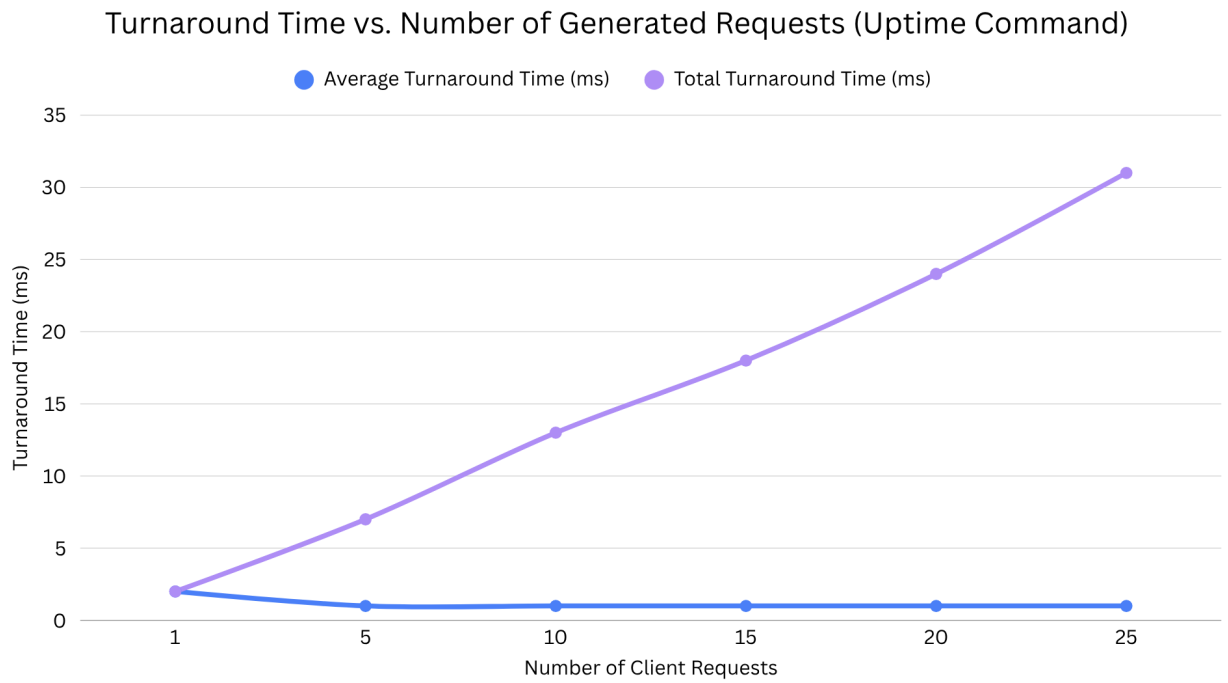
The Iterative Server was tested in a simple, yet thorough manner: print statements. Throughout development, I meticulously crafted test values, and progression messages that would inform me of both the nature of the program’s behaviors, as well as the location of each error within the code. At the same time, I was careful to create the program in self-contained modules. This is why various helper methods are present within my code. I worked on each individual piece of functionality until it was as verbose as it needed to be for the final project execution.

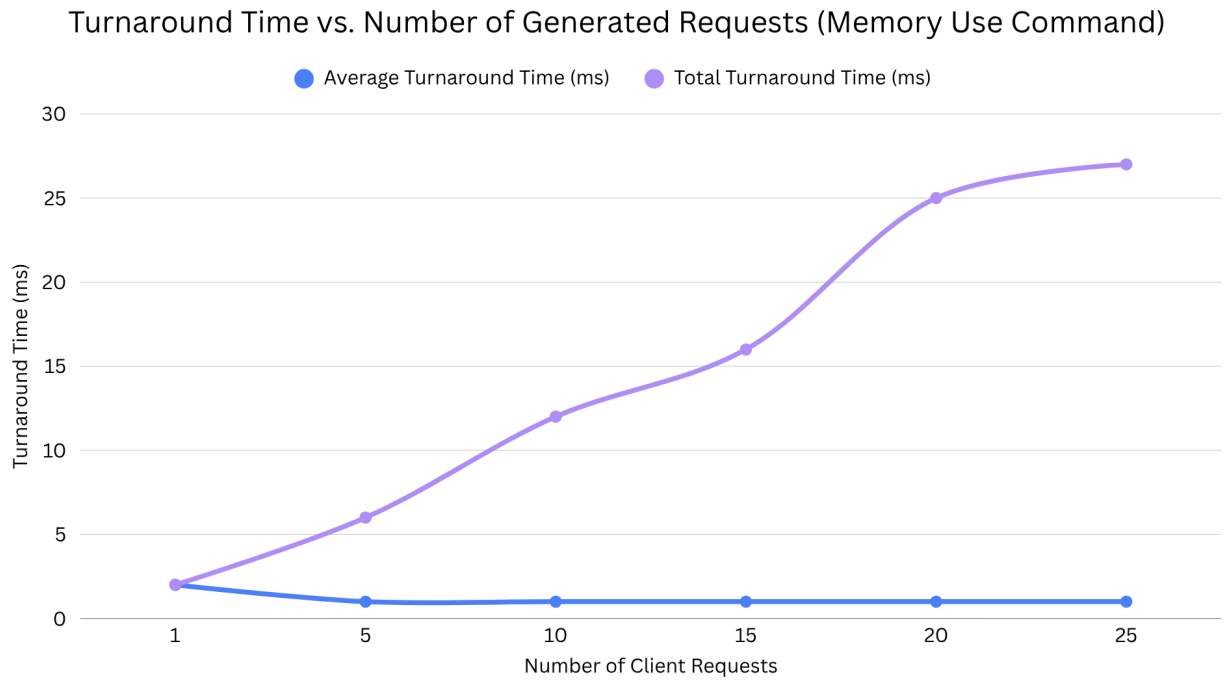
As mentioned previously there were 6 different queries available to the user which provided them with the following data: the current date, server uptime, memory usage, network statistics, users, and running processes. The following commands were utilized to provide this functionality: “date”, "uptime -p,” “cat /proc/meminfo,” “netstat -atun,” “users,” and “ps -e.” These commands are in order as I listed the data above. In addition to providing the user with that data, I collected the individual turnaround times, average turnaround times, and total turnaround times for the thread executions. This was done by wrapping the java…exec methods in a millisecond counter. Below is a collection of the average turnaround times, as well as their

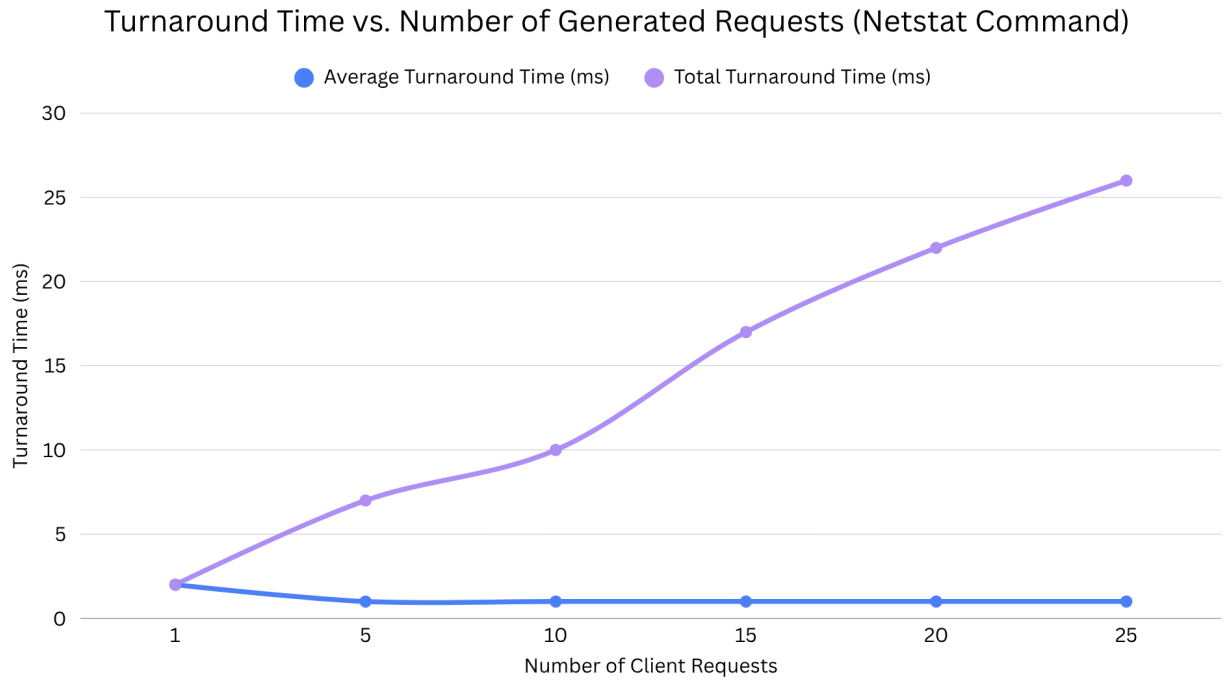
relationships with the number of sessions selected as performed on my M4 MacBook Air via a provided server for educational purposes:

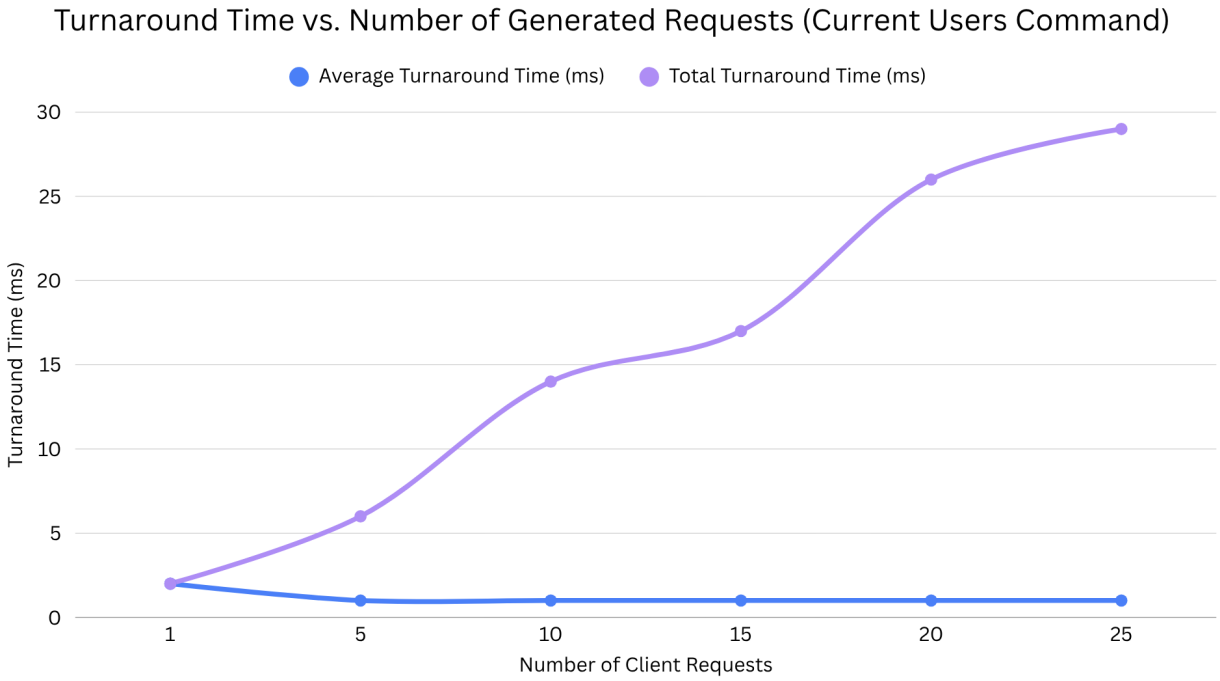


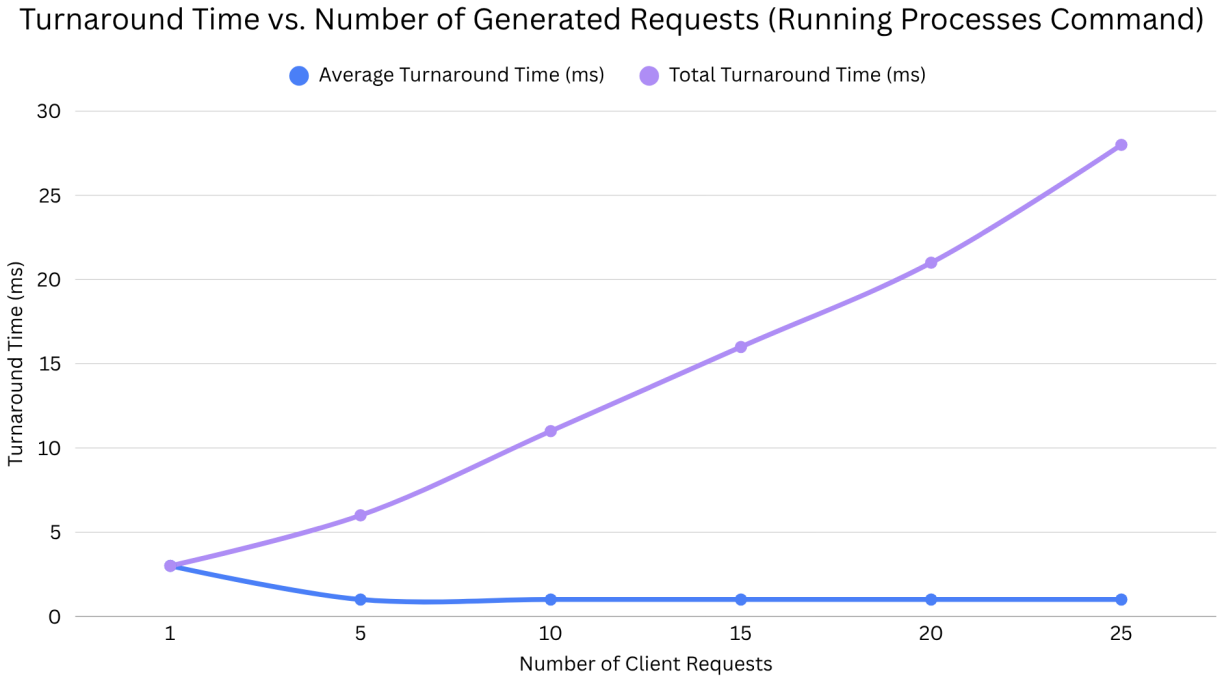

Turnaround Time vs. Number of Generated Requests (Date Command)
Average Turnaround Time (ms)
Total Turnaround Time (ms)
Turnaround Time (ms)
0
5
10
15
20
25
30
1
5
10
15
20
25
Number of Client Requests


Turnaround Time vs. Number of Generated Requests (Uptime Command)
Average Turnaround Time (ms)
Total Turnaround Time (ms)
Turnaround Time (ms)
0
5
10
15
20
25
30
35
1
5
10
15
20
25
Number of Client Requests


Turnaround Time vs. Number of Generated Requests (Memory Use Command)
Average Turnaround Time (ms)
Total Turnaround Time (ms)
Turnaround Time (ms)
30
25
20
15
10
5
0
1
5
10
15
20
25
Number of Client Requests


Turnaround Time vs. Number of Generated Requests (Netstat Command)
Average Turnaround Time (ms)
Total Turnaround Time (ms)
Turnaround Time (ms)
30
25
20
15
10
5
0
1
5
10
15
20
25
Number of Client Requests


Turnaround Time vs. Number of Generated Requests (Current Users Command)
Average Turnaround Time (ms)
Total Turnaround Time (ms)
Turnaround Time (ms)
0
5
10
15
20
25
30
1
5
10
15
20
25
Number of Client Requests


Turnaround Time vs. Number of Generated Requests (Running Processes Command)
Average Turnaround Time (ms)
Total Turnaround Time (ms)
Turnaround Time (ms)
0
5
10
15
20
25
30
1
5
10
15
20
25
Number of Client Requests

Data Analysis

As you can see, my results across the individual commands were astonishingly similar. Initially, this was a big point of concern as much larger differences in times were expected. However, I have to consider that given my machine and the time of day, it is likely that my program put lower than normal stress on the server, perhaps to fewer active users. However, it is easy to notice the initial spike in turnaround time right as I begin querying. As tested on both the educational server, and my personal server, it was common for the first query to take a longer time to complete compared to subsequent attempts.

Based on the data presented, it does not seem like increasing the number of clients has an affect on the individual turn-around times. However, we can acknowledge the positive linear relationship that increasing the number of clients has on the total turnaround time for all of the processes. In most instances, the increasing number of clients increases the total turnaround time by the same factor ~+5milliseconds per 5 clients. The primary cause for the affect on the individual client turnaround seems to be the load each query puts on the cpu. Given the circumstances, these seem to be very light commands on my device. The same can be said for the average turnaround time.

Conclusion and Lessons Learned

Based on the data collected, I can draw the conclusion that the turnaround time for the client-server interactions heavily depends on the load to both the client cpu and the server. There were many lessons learned during the creation of these programs. Specifically: the difficulties of properly communication between client-server programs, token collection and organization, execution of console-level commands via a higher level interface, and many issues with regards to thread synchronization. The largest challenges I overcame had to deal with tokenization, and finding the proper synchronization technique for my situation. Since I implemented the client-server communication linearly, I did not have a convenient buffer to sort between input and output; my threads manually handled the token consumption at both the client and server levels. Likewise, displaying the result of the threads caused initial issues with the recursion of the user menu. These two lessons alone will help me greatly in the planning of my future projects.